

R3.04 : Design patterns

Patrons de conception

Arnaud Lanoix Brauer
Arnaud.Lanoix@univ-nantes.fr



IUT Nantes
Pôle Sciences et technologie

Nantes Université

Département informatique

- 1 Introduction
- 2 Patrons de conception créateur

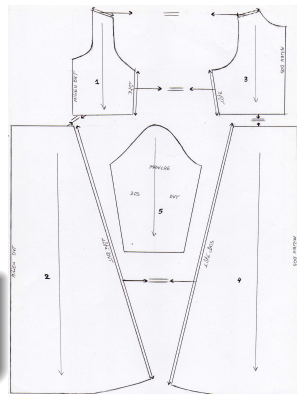
Patrons de conception / Design patterns

Un patron de conception (= **Design Pattern**) est une (la meilleure) *solution de conception* à un problème récurrent, qui pourra être **réutilisée** et **adaptée** indéfiniment

- Analogie = **les patrons en couture** : modèle/plan à réutiliser/adapter
- formalisent des **bonnes pratiques** :
 - ▶ "ne pas ré-inventer la roue"
 - ▶ bénéficier de l'expérience
 - ▶ faciliter la compréhension (et la réutilisation) du code

En pratique

Vous **utilisez déjà** (sans le savoir) de nombreux patrons de conception



IUT Nantes

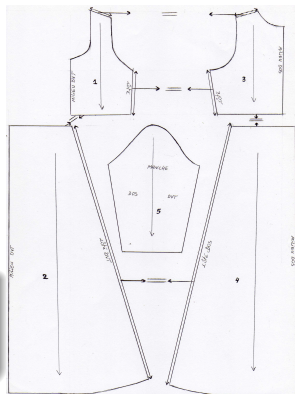
Patrons de conception / Design patterns

Un patron de conception (= **Design Pattern**) est une (la meilleure) *solution de conception* à un problème récurrent, qui pourra être **réutilisée** et **adaptée** indéfiniment

- Analogie = **les patrons en couture** : modèle/plan à réutiliser/adapter
- formalisent des **bonnes pratiques** :
 - ▶ "ne pas ré-inventer la roue"
 - ▶ bénéficier de l'expérience
 - ▶ faciliter la compréhension (et la réutilisation) du code

En pratique

Vous **utilisez déjà** (sans le savoir) de nombreux patrons de conception



IUT Nantes

Les patrons de conception du GoF

Les 23 patrons de conception du GoF

Design Patterns : Elements of Reusable Object-Oriented Software par E. Gamma, R. Helm, R. Johnson et J. Vlissides (= le gang des 4) – 1994.

	Rôle		
Domaine	de construction	Structurel	Comportemental
Classe	Fabrication		Interprète, patron de méthode
Objet	Fabrique , Moniteur, Prototype, Singleton	Adaptateur , Pont, Composite , Décorateur, Façade, Poids-mouche, Procuration	Chaîne de responsabilité, Commande, Itérateur , Médiateur, Memento, Observateur , Etat, Stratégie , Visiteur

Patrons (de conception)

La notion de "patron" peut intervenir à **différents niveaux** :

- Patrons d'analyse
- **Patrons de conception**
- Patrons d'architecture
- Patrons d'implémentation
- ...
- **Anti-patterns** = erreurs courantes de conception/implémentation
<https://fr.wikipedia.org/wiki/Antipattern>
- **Green-patterns** = patrons d'éco-conception / éco-développement
- **Security patterns** = patrons pour assurer la sécurité
- ...

Patrons (de conception)

La notion de "patron" peut intervenir à **différents niveaux** :

- Patrons d'analyse
- **Patrons de conception**
- Patrons d'architecture
- Patrons d'implémentation
- ...
- **Anti-patterns** = erreurs courantes de conception/implémentation
<https://fr.wikipedia.org/wiki/Antipattern>
- **Green-patterns** = patrons d'éco-conception / éco-développement
- **Security patterns** = patrons pour assurer la sécurité
- ...

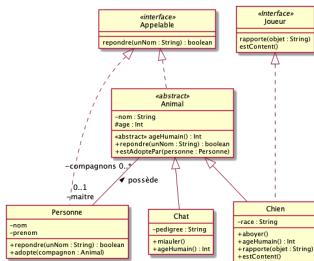
Patrons de conception vs. interfaces

POO vs. Design Patterns

Les design patterns reposent sur les concepts de la POO : en particulier l'héritage et le **polymorphisme**

Interfaces et/ou classes abstraites

Pour mettre en oeuvre des design patterns, une des clefs est de **séparer** l'**interface** (quoi ?) de l'**implémentation** (comment ?)



Patrons de conception : langage(s)

Pattern vs. code

Les design patterns sont **indépendants** des langages de conception/programmation **objet** utilisés

Dans R3.04

- la conception se fera en **UML** (diagramme de classes, principalement)
- la mise ne oeuvre se fera en **Kotlin**

Autres ressources

Vous aurez l'occasion d'utiliser/implémenter des design patterns en Java, en JavaScript, etc.

Patrons de conception : langage(s)

Pattern vs. code

Les design patterns sont **indépendants** des langages de conception/programmation **objet** utilisés

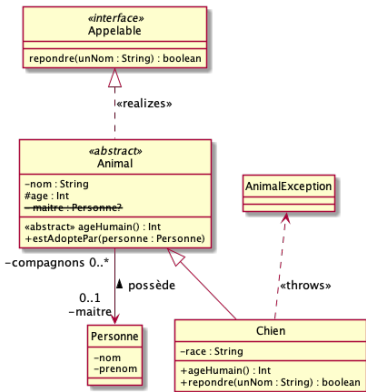
Dans R3.04

- la conception se fera en **UML** (diagramme de classes, principalement)
- la mise ne oeuvre se fera en **Kotlin**

Autres ressources

Vous aurez l'occasion d'utiliser/implémenter des design patterns en Java, en JavaScript, etc.

Diagrammes de classes UML corrects

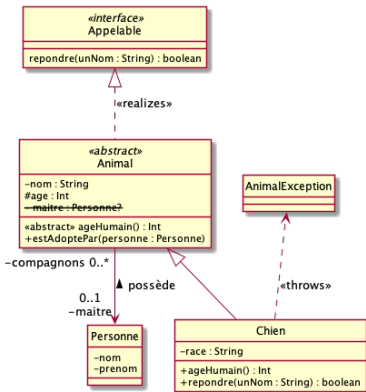


- Classe = forme nominale avec une majuscule
- **Jamais** d'attribut direct correspondant à une classe
- Association = forme verbale (**obligatoire**)
 - ▶ Trait plein
 - ▶ Cardinalités (des deux cotés) **obligatoires**
 - ▶ Navigabilité et rôle **facultatifs**
- Héritage : trait plein + triangle du côté abstrait
 - ▶ **jamais** de forme verbale
- Réalisation : pointillé + triangle, étiqueté `«realizes»`
 - ▶ **toujours** vers une interface
- Dépendance (faible) : trait en pointillé avec une flèche
 - ▶ = "simple utilisation"
 - ▶ étiqueté `«throws»` ou `«uses»`, etc.

ATTENTION

Un diagramme de classes ne respectant **TOUTES** ces consignes ne vaut rien (= 0)

Diagrammes de classes UML corrects

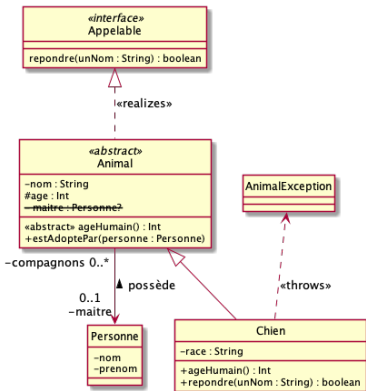


- Classe = forme nominale avec une majuscule
- **Jamais** d'attribut direct correspondant à une classe
- Association = forme verbale (**obligatoire**)
 - ▶ Trait plein
 - ▶ Cardinalités (des deux cotés) **obligatoires**
 - ▶ Navigabilité et rôle **facultatifs**
- Héritage : trait plein + triangle du côté abstrait
 - ▶ **jamais** de forme verbale
- Réalisation : pointillé + triangle, étiqueté «realizes»
 - ▶ **toujours** vers une interface
- Dépendance (faible) : trait en pointillé avec une flèche
 - ▶ = "simple utilisation"
 - ▶ étiqueté «throws» ou «uses», etc.

ATTENTION

Un diagramme de classes ne respectant **TOUTES** ces consignes ne vaut rien (= 0)

Diagrammes de classes UML corrects

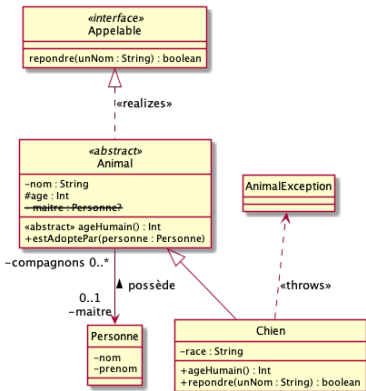


- Classe = forme nominale avec une majuscule
- **Jamais** d'attribut direct correspondant à une classe
- Association = forme verbale (**obligatoire**)
 - ▶ Trait plein
 - ▶ Cardinalités (des deux cotés) **obligatoires**
 - ▶ Navigabilité et rôle **facultatifs**
- Héritage : trait plein + triangle du côté abstrait
 - ▶ **jamais** de forme verbale
- Réalisation : pointillé + triangle, étiqueté «realizes»
 - ▶ **toujours** vers une interface
- Dépendance (faible) : trait en pointillé avec une flèche
 - ▶ = "simple utilisation"
 - ▶ étiqueté «throws» ou «uses», etc.

ATTENTION

Un diagramme de classes ne respectant **TOUTES** ces consignes ne vaut rien (= 0)

Diagrammes de classes UML corrects

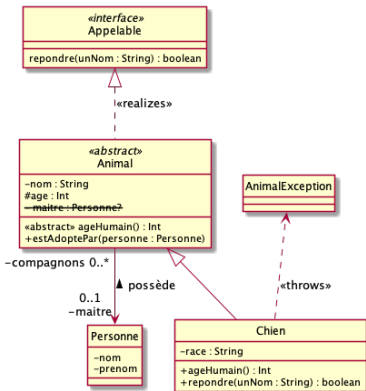


- Classe = forme nominale avec une majuscule
- **Jamais** d'attribut direct correspondant à une classe
- Association = forme verbale (**obligatoire**)
 - ▶ Trait plein
 - ▶ Cardinalités (des deux cotés) **obligatoires**
 - ▶ Navigabilité et rôle **facultatifs**
- Héritage : trait plein + triangle du côté abstrait
 - ▶ **jamais** de forme verbale
- Réalisation : pointillé + triangle, étiqueté `«realizes»`
 - ▶ **toujours** vers une interface
- Dépendance (faible) : trait en pointillé avec une flèche
 - ▶ = "simple utilisation"
 - ▶ étiqueté `«throws»` ou `«uses»`, etc.

ATTENTION

Un diagramme de classes ne respectant **TOUTES** ces consignes ne vaut rien (= 0)

Diagrammes de classes UML corrects

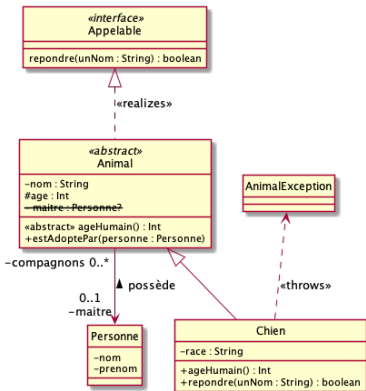


- Classe = forme nominale avec une majuscule
- **Jamais** d'attribut direct correspondant à une classe
- Association = forme verbale (**obligatoire**)
 - ▶ Trait plein
 - ▶ Cardinalités (des deux cotés) **obligatoires**
 - ▶ Navigabilité et rôle **facultatifs**
- Héritage : trait plein + triangle du côté abstrait
 - ▶ **jamais** de forme verbale
- Réalisation : pointillé + triangle, étiqueté `«realizes»`
 - ▶ **toujours** vers une interface
- Dépendance (faible) : trait en pointillé avec une flèche
 - ▶ = "simple utilisation"
 - ▶ étiqueté `«throws»` ou `«uses»`, etc.

ATTENTION

Un diagramme de classes ne respectant **TOUTES** ces consignes ne vaut rien (= 0)

Diagrammes de classes UML corrects

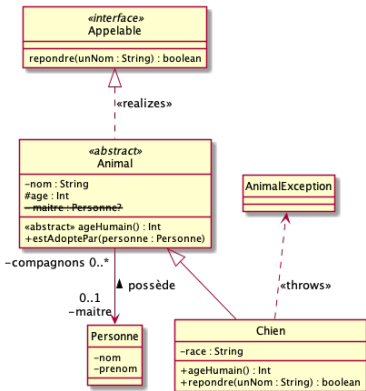


- Classe = forme nominale avec une majuscule
- **Jamais** d'attribut direct correspondant à une classe
- Association = forme verbale (**obligatoire**)
 - ▶ Trait plein
 - ▶ Cardinalités (des deux cotés) **obligatoires**
 - ▶ Navigabilité et rôle **facultatifs**
- Héritage : trait plein + triangle du côté abstrait
 - ▶ **jamais** de forme verbale
- Réalisation : pointillé + triangle, étiqueté `«realizes»`
 - ▶ **toujours** vers une interface
- Dépendance (faible) : trait en pointillé avec une flèche
 - ▶ = "simple utilisation"
 - ▶ étiqueté `«throws»` ou `«uses»`, etc.

ATTENTION

Un diagramme de classes ne respectant **TOUTES** ces consignes ne vaut rien (= 0)

Diagrammes de classes UML corrects



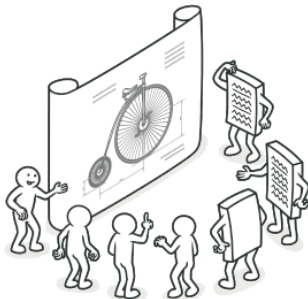
- Classe = forme nominale avec une majuscule
- **Jamais** d'attribut direct correspondant à une classe
- Association = forme verbale (**obligatoire**)
 - ▶ Trait plein
 - ▶ Cardinalités (des deux cotés) **obligatoires**
 - ▶ Navigabilité et rôle **facultatifs**
- Héritage : trait plein + triangle du côté abstrait
 - ▶ **jamais** de forme verbale
- Réalisation : pointillé + triangle, étiqueté «realizes»
 - ▶ **toujours** vers une interface
- Dépendance (faible) : trait en pointillé avec une flèche
 - ▶ = "simple utilisation"
 - ▶ étiqueté «throws» ou «uses», etc.

ATTENTION

Un diagramme de classes ne respectant **TOUTES** ces consignes ne vaut rien (= 0)

Références / remerciements

- E. Gamma, R. Helm, R. Johnson et J. Vlissides : *Design Patterns : Elements of Reusable Object-Oriented Software*
- A. Shalloway et J.R. Trott : *Design patterns par la pratique*
- A. Soshin : *Kotlin Design Patterns and Best Practices*
- D. Tamzalit pour son cours et ses TDs/TPs à propos des DP
- T. Béziers La Fosse pour son cours et ses TDs/TPs à propos des DP
- Wikipedia : https://fr.wikipedia.org/wiki/Patron_de_conception
- Refactoring Guru : <https://refactoring.guru/fr/design-patterns>¹

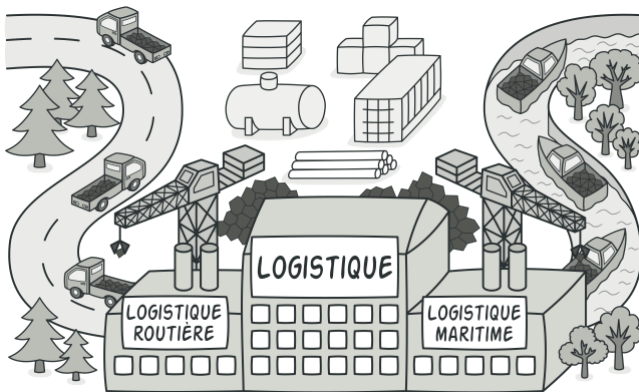


1. Merci également pour les images "humoristiques"

1 Introduction

2 Patrons de conception créateur

Patron Fabrique – Factory pattern



Patron Fabrique – Factory pattern

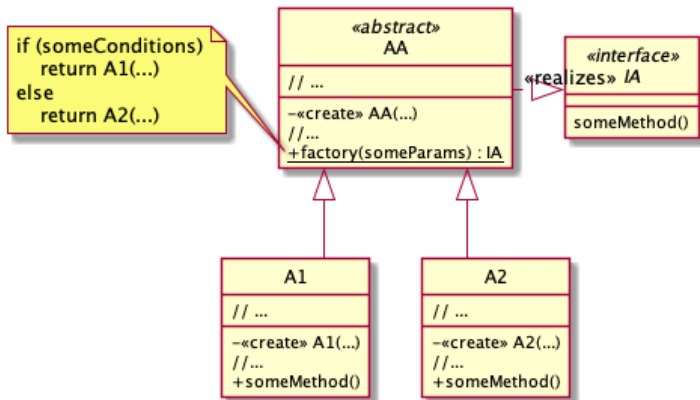
Problème

- Comment **construire** un objet à partir de **paramètres complexes** ?
- Comment **vérifier** les paramètres **avant** de construire l'objet ?
- Quelle(s) classe(s) **instancier** parmi une hiérarchie de classes ?

Solution

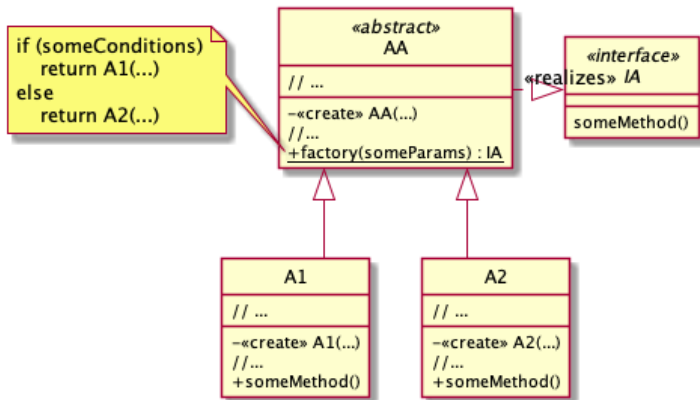
La **fabrique** fournit une (unique) méthode (statique) qui va retourner une **instance de classes** (parmi plusieurs sous-classes possibles) **en fonction** des paramètres

Patron Fabrique : schéma UML



- L'interface `IA` n'est pas obligatoire = fusionnable avec `A`
- En utilisant `factory(...)` la classe exacte instanciée n'est donc pas connue

Patron Fabrique : schéma UML



- L'interface `IA` n'est pas obligatoire = fusionnable avec `A`
- En utilisant `factory(...)` la classe exacte instanciée n'est donc pas connue

Patron Fabrique – précisions

- Une bonne idée peut être est de rendre `private` le(s) constructeur(s) des classes considérées
 - ▶ Rendre **obligatoire** l'utilisation de la Fabrique
- Fabrique permet de **séparer** la **création** des objets de leur **utilisation**.
- Fabrique fourni des **noms plus lisibles** / **auto-documentées** que les constructeurs.
- Plusieurs fabriques peuvent être regroupées en une **fabrique abstraite**.
- Fabrique est souvent utilisé conjointement à un autre patron de conception.

pour aller plus loin : Builder

Le patron de conception **Monteur (Builder)** est assez proche, mais dédié à la **composition** d'objets complexes

Patron Fabrique – précisions

- Une bonne idée peut être est de rendre `private` le(s) constructeur(s) des classes considérées
 - ▶ Rendre **obligatoire** l'utilisation de la Fabrique
- Fabrique permet de **séparer** la **création** des objets de leur **utilisation**.
- Fabrique fourni des **noms plus lisibles** / **auto-documentées** que les constructeurs.
- Plusieurs fabriques peuvent être regroupées en une **fabrique abstraite**.
- Fabrique est souvent utilisé conjointement à un autre patron de conception.

pour aller plus loin : Builder

Le patron de conception **Monteur (Builder)** est assez proche, mais dédié à la **composition** d'objets complexes

Exemple de fabrique (1)

```
abstract class Animal(  
    val nom : String,  
    val age : Int = 1) {  
  
    companion object {  
        fun fabrique(sorteAnimal : String,  
            nom : String,  
            age : Int) : Animal {  
            return when (sorteAnimal) {  
                "chien" -> Chien(nom, age)  
                "chat" -> Chat(nom, age)  
                else -> throw AnimalException()  
            }  
        }  
    }  
}
```

```
var monAnimal : Animal  
monAnimal = Animal.fabrique("chien", "rogue", 2)  
monAnimal = Animal.fabrique("chat", "totoro", 1)  
monAnimal = Animal.fabrique("tortue", "leonard", 80)
```

Gestion des erreurs

Utiliser une fabrique permet de gérer les erreurs de construction, ici une levée d'exception

Exemple de fabrique (2)

```
class Complex
private constructor(r : Double, i : Double) {
    private val real = r
    private val imag = i

companion object {
    fun fromCartesian(real : Double, imag : Double) {
        return Complex(real, imag)
    }
    fun fromPolar(rho : Double, theta : Double) {
        return Complex(rho * Math.cos(theta), rho * Math.sin(theta))
    }
}
```

- Le constructeur est `private`, ce qui oblige l'utilisation des fabriques

Kotlin

Les méthodes `listOf(...)`, `mutableListOf(...)`, etc. sont des fabriques

Exemple de fabrique (2)

```
class Complex
private constructor(r : Double, i : Double) {
    private val real = r
    private val imag = i

companion object {
    fun fromCartesian(real : Double, imag : Double) {
        return Complex(real, imag)
    }
    fun fromPolar(rho : Double, theta : Double) {
        return Complex(rho * Math.cos(theta), rho * Math.sin(theta))
    }
}
```

- Le constructeur est `private`, ce qui oblige l'utilisation des fabriques

Kotlin

Les méthodes `listOf(...)`, `mutableListOf(...)`, etc. sont des fabriques